

Does a Context File Actually Help?

An Empirical, Task-Level Comparison of Agentic Coding With and Without an AGENT.md Context File

Case study on the UniDesk fullstack ticketing application (FastAPI + React)

Jannatul Ferdous Nabila

Topic:1

Abstract

Context files like AGENT.md are widely recommended for agentic coding, but rarely measured. This paper tests the claim on one full-stack repository ([UniDesk](#), FastAPI/React) across five tasks of increasing size and ambiguity, each attempted once without a context file and once with one, scored against a rubric of correctness, convention adherence, unrequested extras, and edge-case handling using git history as evidence. The size hypothesis holds, but not for the assumed reason: small tasks converge regardless of condition, while on large tasks the context file's value was governance, not intelligence — it prevented an orphaned migration, an unapproved endpoint, and two silent regressions that shipped past a green test suite. Critically, the context-file run was not strictly better: it was twice more conservative than required, and once missed a check the free-form version caught. Both wins and regressions are reported.

Contents

1. Introduction.....	4
2. The Question, and Why It Matters.....	4
3. Methodology.....	4
4. Results.....	5
4.1 Small tasks: convergence, not improvement	6
4.2 Task 3: ambiguity resolved instead of guessed	6
4.3 Task 4: governance prevented compounding damage.....	6
4.4 Task 5: the failures a green test suite does not catch.....	7
5. What Surprised Me	9
6. Limitations	10
7. Recommendations.....	10
8. Conclusion	11
Appendix.....	11
Prompts Used (Task Descriptions Given to the Agent)	11

1. Introduction

Agentic coding assistants are increasingly guided by written context files — AGENT.md, CLAUDE.md, AGENTS.md — that encode project conventions, process rules, and approval gates. Despite their popularity, the actual effect of these files on code quality is rarely tested empirically; most claims rest on developer intuition rather than controlled comparison. This paper investigates that gap using a single case study: the [UniDesk](#) fullstack ticketing application. Five development tasks, ranging from a one-line UI fix to a security-hardening pass, were each implemented twice — once without a context file and once with an AGENT.md file present — and the two outputs were compared against a fixed rubric covering correctness, convention adherence, unrequested extras, and edge-case handling. The aim is not to confirm that context files help in the abstract, but to establish where, how, and by how much they help, and where they do not. The remainder of the paper presents the hypothesis, the method, the task-by-task results, the surprises, the study's limitations, and practical recommendations.

2. The Question, and Why It Matters

“Context files help” is close to gospel in agentic coding circles. AGENT.md, CLAUDE.md, and AGENTS.md files are recommended almost reflexively, on the theory that giving a coding agent standing instructions — how to read the spec, which files own which concerns, what counts as an approved change — will produce better code. What is rarely established is whether this is true, and if it is, where the benefit actually shows up. Two very different mechanisms could produce the same anecdotal impression of “it helps”: the agent could be writing objectively better code, or the agent could simply be making fewer unsupervised decisions, each individually small, that compound into damage only on larger and more ambiguous tasks.

These are not the same claim, and they lead to different advice. If context files make an agent smarter, every task benefits proportionally to its size. If context files mainly constrain scope and reduce unsupervised inference, then trivial tasks should show almost no benefit — there is nothing to over-infer on a one-line text change — while large, ambiguous, security- or schema-adjacent tasks should show a large benefit, because that is where an agent has the most room to guess wrong, and where a wrong guess is expensive to reverse.

This distinction matters practically, not just academically. Writing and maintaining a context file has a real cost: someone has to author it, and — as this study will show — someone has to keep it synchronized with a codebase that keeps changing underneath it. If the benefit is concentrated in large, ambiguous, high-blast-radius work, that is a very different investment case than if the benefit is spread evenly across everything an agent touches. This paper's hypothesis, stated before any task was scored: context files produce a strong, measurable effect on large or ambiguous tasks and a negligible effect on small, well-specified ones. The null hypothesis is that the effect is roughly constant across task size, which would suggest the context file is mostly acting as a general quality multiplier rather than a scope-and-governance tool.

3. Methodology

The experiment was run against one repository, [UniDesk](#), a support-ticketing application with a FastAPI backend, a PostgreSQL database via SQLAlchemy/Alembic, and a React/Tailwind frontend. Using a single codebase trades external validity for depth: every task shares the same conventions, the same existing test suite, and the same real database state, which makes it possible to trace concrete artifacts — an orphaned migration, a wrong status code — rather than relying on subjective read of “code quality.”

Five tasks were selected to span a deliberate range of size and ambiguity:

Task 1 — Improve Empty Dashboard Text. A one-line UI copy change in the dashboard empty state. Minimal scope, minimal ambiguity.

Task 2 — Add Dashboard Ticket Search. A client-side search feature that has to coexist with an existing status filter and introduce a second, distinct empty state without disturbing the first.

Task 3 — Enforce Ticket Lifecycle Transitions. A backend business-rule task with no written specification of which status transitions are legal, forcing the agent to infer policy rather than just implement it.

Task 4 — Ticket Assignment and Agent Work Queues. A full-stack feature: a new relationship and migration, a protected endpoint, role-based permissions, and corresponding dashboard filters and UI controls.

Task 5 — Harden Authentication and Session Management. A cross-cutting security pass touching JWT validation, centralized role dependencies, startup configuration validation, and frontend session handling.

Each task was run under two conditions on the same starting codebase: once with no context file present, and once with an AGENT.md file available that specified process (read spec.md first, inspect existing behavior before editing, prefer the smallest change, ask before altering public API paths or error messages, run focused tests before the full suite). Outputs were compared against a rubric fixed in advance of scoring, defined below.

Criterion	What it measures
Correctness	Does the change satisfy the stated requirement, with no functional defects?
Convention adherence	Does it match the codebase's existing patterns — naming, structure, error handling, migration style?
Unrequested extras	Did the agent add scope, endpoints, or behavior beyond what was asked, without approval?
Edge-case handling	Are failure paths, invalid states, and boundary conditions handled explicitly rather than left to chance?

Table 01: Evaluation Rubric Criteria

4. Results

Table 2 summarizes how each task scored against the four rubric criteria, under both conditions. Detailed evidence for each row follows in the subsections below.

Task	Condition	Correctness	Convention	Unrequested extras	Edge cases
1. Empty-state text	Without	Met, after redefinition	Met	None	N/A (trivial)
	With	Met	Met	None	N/A (trivial)
2. Dashboard search	Without	Met	Good	None	Met
	With	Met	Strong	None	Met, resolved faster
3. Lifecycle rules	Without	Partial policy invented	Mixed	Scope invented on the spot	Ad hoc
	With	Met	Strong	None	Explicit (409 Conflict)

Task	Condition	Correctness	Convention	Unrequested extras	Edge cases
4. Assignment module	Without	Met, but unstable	Weak — orphaned migration	Unapproved endpoint added	Partial
	With	Met, more conservative	Strong	None (approval-gated)	Strong, but permission broadened beyond spec
5. Auth hardening	Without	Failed — 2 silent regressions	Weak	None	Failed — role/404 ordering
	With	Met	Strong	None	Strong, after one initial gap closed

Table 02: Task Evaluation Summary Across Conditions

4.1 Small tasks: convergence, not improvement

On Task 1, the code produced with and without AGENT.md was nearly identical — a single string replaced in the dashboard's empty-state component. For this tiny Task 1 and it is a one line tiny task so AGENT.md did not make the coding itself significantly better. The change was simple enough either way. The rubric shows equal correctness and convention adherence in both conditions; the only column that moved was process efficiency.

Task 2 shows the same pattern at slightly larger scale. Both conditions converged on a client-side, debounced search filter that coexists with the status tabs. The context-file run resolved several small design questions faster and with more confidence — whether to split search into its own component, which fields to search, how to distinguish a “no tickets in this view” empty state from a “no search results” empty state — but arrived at essentially the same feature. Correctness and edge-case handling scored equivalently; convention adherence and decision latency favored the AGENT.md run.

4.2 Task 3: ambiguity resolved instead of guessed

Lifecycle enforcement is where the effect starts to separate. Without a specification of valid status transitions, the pre-AGENT.md attempt had to invent business policy on the spot — deciding, without authority to do so, whether resolved-to-in_progress or closed-to-open should be legal. With AGENT.md those business policy is already been declared in AGENT.md file, so the task became implementation and verification rather than policy invention.

4.3 Task 4: governance prevented compounding damage

The pre-AGENT.md attempt (commit 8cf4a5c) produced syntactically correct, logically coherent code — this was never a broken-code problem — but it left two artifacts that outlived the commit itself. First, its migration `a1b2c3d4e5f6_add_ticket_assignment.py` was applied against the real local Postgres database and then the commit was reverted from git without the database being rolled back. Second, it added a new public endpoint, `GET /api/v1/users/support-agents`, with no approval gate — the exact class of unreviewed surface-area growth that AGENT.md's later “ask before changing public API paths” rule appears to exist specifically to prevent.

The comparison is summarized below.

Aspect	Without AGENT.md	With AGENT.md
New public endpoint	Added, unreviewed	None added — approval required first
Reassignment UI	Direct picker of any agent	Self-assign / take-over / release only
Release permission	Own assignment only	Any agent may release any assignment
Dashboard filter	Client-side JS filter	Server-side SQL filter via query param
Tests	7, folded into test_rbac.py	16, dedicated test_assignment.py
Migration state	Applied, then orphaned by revert	Reconciled cleanly against the orphaned state

Table 03: Detailed Comparison of Task 4 Implementation Approaches

Two rows in that table are reported as regressions, not improvements, because the honest comparison requires it. The original prompt said agents should be able to “release their own assignment”; the pre-AGENT.md attempt implemented exactly that, while the AGENT.md-guided version broadened release permission to any agent on any ticket, reasoning by analogy to existing status/priority permissions a defensible design call, but a looser reading of the actual requirement. Similarly, because AGENT.md required approval before adding a new endpoint, the disciplined version cannot support a direct “assign to a specific named colleague” action, which the free-form version's dropdown UI handled better. AGENT.md made the output safer and more consistent with governance, not strictly more capable.

4.4 Task 5: the failures a green test suite does not catch

The authentication-hardening task shows a different and, in a production sense, more dangerous failure mode than Task 4: the pre-AGENT.md commit (db536e9) was not obviously broken. It ran. Its own test suite passed. The defects were two silent behavioral regressions that no test in either suite happened to exercise. First, a single hardcoded-message `require_support_agent` dependency was reused across two endpoints; correct for the status-update endpoint, it produced a wrong, confusing 403 message (“...update ticket status or priority”) on the assignment endpoint, which AGENT.md explicitly classifies as an API-contract change requiring approval. Second, wiring `require_employee` in as a FastAPI dependency changed evaluation order: dependencies resolve before the function body runs, so a nonexistent-ticket check that previously guaranteed a 404 regardless of role now returned a role-based 403 first, with a message about ticket creation on what was actually an update call — the wrong status code and the wrong message, invisible for roughly two hours until the commit was reverted, because neither test suite tested “wrong role plus nonexistent ticket” together.

The AGENT.md-guided implementation deliberately did not fully centralize the update/delete-ticket role checks into the same shared dependency, even though a stricter reading of “centralize reusable role dependencies” might suggest it should, specifically because full centralization is what caused this exact class of bug the first time. That is a considered trade-off, not a rule mechanically followed.

This task also contains the study's clearest instance of the free-form version being better: its config validation rejected secret keys that still looked like the literal `<generate-a-random-secret...>` placeholder text from `.env.example`, a real and specific attack the AGENT.md-guided version had not initially guarded against (it sidestepped the issue via a different template design, shipping an empty default rather than a placeholder string — not a substitute for the guard). That check was

ported over and covered with tests during this study, closing the gap, but the point stands: the disciplined process did not catch everything the free-form one caught.

Figure 1: Average Rubric Score by Task and Condition

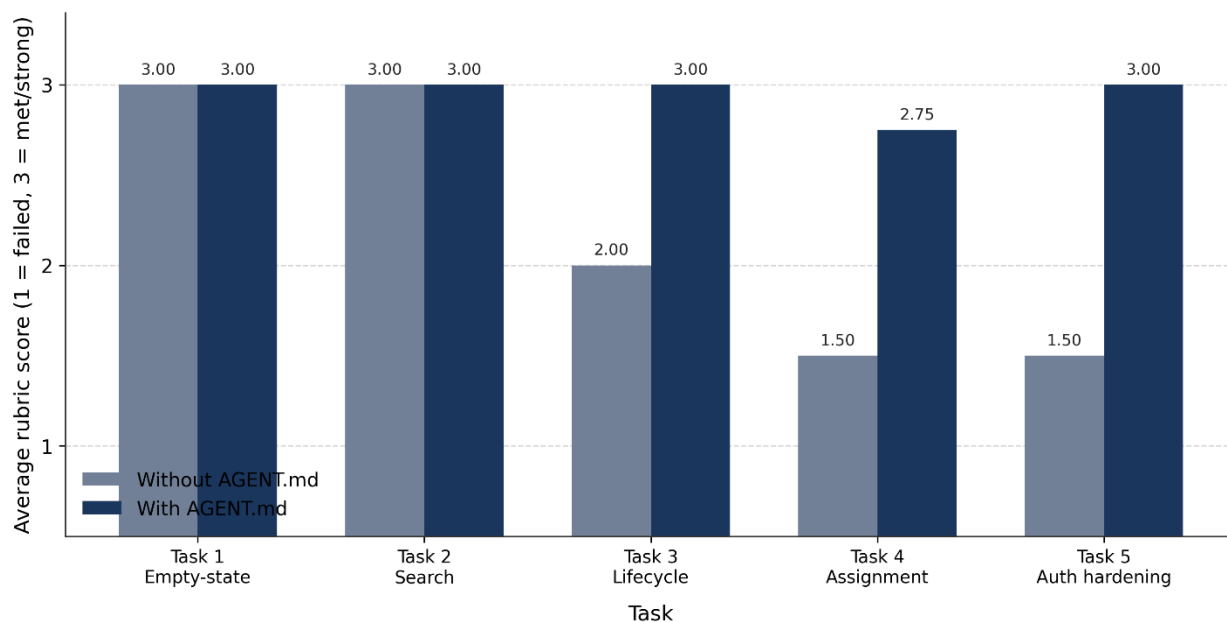


Figure 1. Average rubric score by task and condition. Small tasks (1–2) converge; large tasks (3–5) diverge sharply once AGENT.md is present.

Figure 2: Total Rubric Flags Across All 5 Tasks

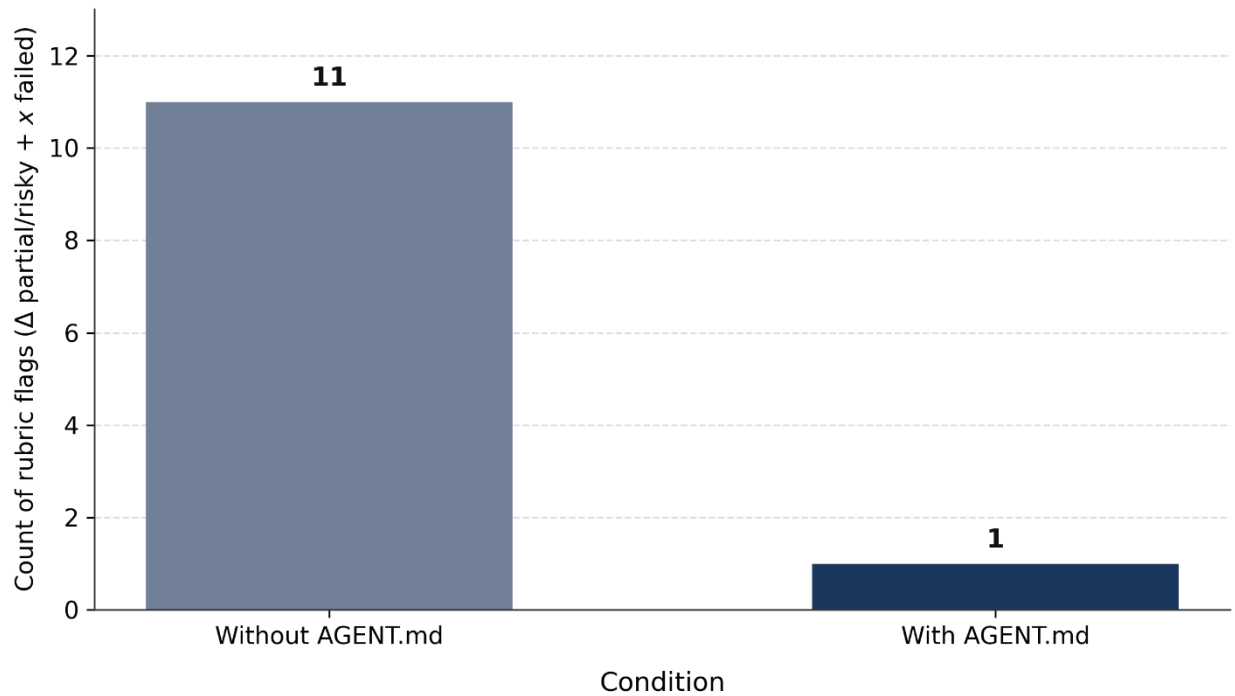


Figure 2. Total rubric flags (Δ partial/risky + $\times$ failed) summed across all 20 criterion-cells (5 tasks $\times$ 4 criteria) scored per condition.

5. What Surprised Me

Four findings were not predicted going into this study.

The size effect is real, but the mechanism is governance, not intelligence

The original hypothesis was that context files would help more on large tasks because there was “more for the agent to get right.” The actual mechanism looks different: on small tasks there is almost nothing to over-infer, so there is nothing for a context file to prevent. On large tasks, the context file's value came almost entirely from constraining scope — no new unapproved endpoint, no full centralization of a risky dependency, an orphaned migration reconciled instead of ignored — not from the agent reasoning better about the core problem. In both large-task cases, the underlying code the agent could write was already within its ability without the file; what the file changed was which parts of that ability got exercised without a human in the loop.

The context-file run was not uniformly better

In Task 4, the disciplined implementation was measurably more conservative on release permissions and reassignment UX than the original prompt asked for. In Task 5, the free-form implementation caught a placeholder-secret attack the disciplined one initially missed. A context file that enforces caution will sometimes produce a narrower feature than a human actually asked for, and reviewers should expect to occasionally loosen a rule the file imposed, not just tighten gaps it left.

6. Limitations

Single repository, single stack. All five tasks were run against one FastAPI/React/Postgres application. Whether the effect generalizes to other languages, frameworks, or architectural styles where existing conventions are less discoverable by inspection is untested.

Single agent as implementer and evaluator. The same model both produced the code under both conditions and, functionally, assessed it in this write-up. There was no independent or blinded reviewer scoring the two outputs against the rubric, which introduces a real risk of self-serving interpretation, particularly in the sections framed as wins for the AGENT.md condition.

Small sample size. Five tasks, two conditions each, is enough to illustrate a pattern but not enough to claim statistical significance. A single differently-worded task could plausibly change which condition looks better on any given rubric row.

One style of context file. AGENT.md here encoded a specific process (read spec, inspect first, minimal diff, ask before touching public contracts). A differently structured context file — shorter, longer, framed as rules versus as narrative might produce a different-sized effect, and this study cannot distinguish “context files help” from “this particular AGENT.md helped.”

7. Recommendations

Treat the context file as a living document with an owner, and should be caught by periodic review, ideally tied to the same PR process that changes the behavior it describes.

Reserve context files for medium-to-large, ambiguous, or high-blast-radius work; do not expect measurable benefit on trivial, well-specified changes, and do not use the presence of a context file as a substitute for writing an unambiguous task description in the first place.

Explicitly enumerate “ask before changing” categories rather than leaving them implicit — this study's clearest wins (blocked unapproved endpoint, preserved error message, preserved status-code ordering) all came from concrete, named categories (public API paths, error messages, dependency evaluation order), not from a general instruction to “be careful.”

Add regression tests targeting the specific silent-failure classes this study found — role-check-versus-404 precedence, and per-endpoint error message text — since both bugs shipped past a fully green test suite and would not be caught by typical happy-path coverage.

Any time a context file forces a more conservative implementation than the literal request (as with the broadened-then-should-have-been-narrower release permission in Task 4), flag the narrowing explicitly to the requester rather than silently under-delivering; caution should be visible, not assumed correct.

Future work should address this study's own limitations directly: a pre-registered rubric, blind scoring by a separate reviewer, matched paired trials for every task (not reconstructed history), and replication across at least one additional stack and one additional agent, to separate “context files help” from “this context file, on this codebase, with this model, helped.”

8. Conclusion

This study set out to test whether a context file measurably improves agentic coding output, and if so, where. The evidence from five tasks on the UniDesk codebase supports a narrower claim than the popular one: context files did not make the coding agent smarter, and on small, well-specified tasks they made almost no measurable difference. Their value emerged only on larger, more ambiguous, higher-risk tasks, where they functioned as governance rather than intelligence closing an orphaned migration, blocking an unapproved endpoint, and preserving correct error messages and status codes that a green test suite failed to protect. At the same time, the disciplined process was not universally superior: it was twice more conservative than the literal request required, and once missed a security check the free-form version caught. Both outcomes matter for how context files should be used in practice as a scoped governance tool for high-blast-radius work, kept current through the same review process as the code it governs, rather than as a blanket guarantee of better output. Future work with blind scoring, matched trials, and additional codebases and agents would help confirm how far this pattern generalizes.

In one sentence: on this codebase, the context file did not make the agent smarter — it made the agent's unsupervised guesses fewer, and on the two tasks large enough for guesses to compound, that difference was the entire story.

Appendix

Prompts Used (Task Descriptions Given to the Agent)

Each task was issued to the coding agent as a short, plain-language run once with no context file and once with AGENT.md available:

Task 1: Improve Empty Dashboard Text

In the UniDesk frontend, update the dashboard empty-state message shown when no tickets match the current filter. Replace the existing message with: “No tickets found for this view.” Keep the change limited to the relevant dashboard component, and do not modify API behavior, routing, or state logic.

Task 2: Add Dashboard Ticket Search

Prompt: Add a ticket search control to the UniDesk dashboard. Users should be able to search tickets by title, description, or creator name, case-insensitively. Search must work together with the existing status filter tabs, update as the user types, and show an appropriate empty state when no tickets match. Keep filtering client-side because the dashboard already loads the ticket list. Follow the existing Tailwind styling and component structure. Do not change the backend API.

Task 3: Enforce Ticket Lifecycle Transitions

Prompt: Implement ticket lifecycle transition validation in the UniDesk backend. Support agents may update a ticket only through these transitions: open -> in_progress, in_progress -> resolved, and resolved -> closed. Allow reopening from resolved to in_progress and from closed to open for correction or follow-up. Reject all other status changes with a clear 409 Conflict response. Priority-only updates should remain valid without changing status. Put the transition logic in a reusable helper or domain utility rather than embedding a large conditional

directly in the router. Add focused pytest coverage for every allowed and rejected transition, while preserving the existing RBAC behavior.

Task 4: Implement Ticket Assignment and Agent Work Queues

Prompt: Implement an end-to-end ticket assignment module for UniDesk. Add an optional `assigned_to` relationship from tickets to support-agent users. Support agents must be able to assign an unassigned ticket to themselves, release their own assignment, and assign or reassign tickets through a protected API endpoint. Employees must not be able to assign tickets. Validate that only users with the `support_agent` role can be assignment targets. Add database migration support, request/response schemas, and pytest coverage for assignment permissions, reassignment, releasing, missing users, and missing tickets.

Update the React application so the dashboard can filter by All, Unassigned, and Assigned to Me, and ticket details show the current assignee. Support agents should have assignment controls; employees should only see read-only assignment information. Preserve all existing ticket, comment, status, and priority behavior.

Task 5: Harden Authentication and Session Management

Prompt: Harden UniDesk authentication and authorization across the backend and frontend.

Backend requirements:

Reject malformed JWTs, missing sub claims, non-integer subjects, expired tokens, and tokens signed with an invalid secret with a consistent 401 Unauthorized response.

Remove the insecure fallback secret. Application startup or configuration loading must fail clearly when a production secret is missing or unsafe.

Centralize reusable role dependencies for employee-only and support-agent-only endpoints.

Treat the database user record as authoritative for the current role; do not trust a stale JWT role claim.

Preserve the existing whitelist, password validation, and ownership rules.

Add security-focused tests for malformed claims, expired tokens, nonexistent users, stale roles, and protected endpoints.

Frontend requirements:

Improve AuthContext and the Axios interceptor so expired or invalid sessions are cleared consistently.

Prevent repeated unauthorized requests from causing redirect loops.

Ensure loading state is resolved for every authentication path.

Keep the existing route protection and login/register UX intact. Make the changes in a maintainable way, document any unavoidable local-development configuration requirements, and avoid exposing tokens or secrets in logs.

UniDesk Application Live Link: [UniDesk](#)